Mini Project

on

Aadhaar Enrollment Forecasting System

State-Level Daily Enrollment Prediction using Machine Learning & Deep Learning

As a part of

Data Science (CE0630)

Submitted by

Kunj Shah (IU2341230095)

Krish Patel (IU2341230112)

Keshav Khatri (IU2341230068)

In the partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

INDUS UNIVERSITY

WHERE PRACTICE MEETS THEORY

NAAC ACCREDITED

INSTITUTE OF TECHNOLOGY & ENGINEERING, INDUS UNIVERSITY
UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA
WEB: www.indusuni.ac.in

MAY 2026

PROBLEM STATEMENT: AADHAAR ENROLLMENT FORECASTING SYSTEM

Abstract

This project develops a machine learning system to forecast daily Aadhaar enrollment volumes at the state level across India. Accurate enrollment prediction enables the Unique Identification Authority of India (UIDAI) and state governments to allocate enrollment infrastructure efficiently, plan staffing, and detect anomalous enrollment patterns indicative of data quality issues.

The system processes multi-modal time-series data sourced from the UIDAI public API — encompassing enrollment counts disaggregated by age group, demographic update records, and biometric update records — spanning March 2025 to December 2025 across 52 state-level administrative units. A comprehensive feature engineering pipeline extracts 49 predictive features including temporal lag sequences, rolling statistics, cyclical calendar encodings, holiday proximity indicators, and census-derived per-capita normalization features.

Five machine learning models are trained and evaluated on a strict temporal 80/20 train-test split to prevent future data leakage: Ridge Regression, Random Forest, XGBoost, LightGBM, and a custom PyTorch LSTM network. A stacked ensemble combining the top-performing models achieves the best generalization, with a test R^2 of 0.3817 and RMSE of 1,437 enrollments per state per day. The project is deployed as a five-tab interactive Streamlit dashboard with live inference, model benchmarking, anomaly detection, and MLOps drift monitoring capabilities.

Introduction

Data Science is a field that combines statistical analysis, machine learning, and data processing techniques to extract meaningful insights from large datasets and solve real-world problems through data-driven decision making.

The Aadhaar system, administered by the Unique Identification Authority of India (UIDAI), is the world's largest biometric identity program with over 1.4 billion enrolled residents. Daily enrollment operations generate high-frequency, state-level count data that exhibits strong temporal autocorrelation, seasonal patterns driven by government drives and public holidays, and extreme scale heterogeneity — Uttar Pradesh enrolls over 3,000 individuals per day on average, while Lakshadweep enrolls fewer than one.

Traditional approaches to enrollment planning rely on historical averages and manual judgement, which fail to capture day-of-week effects, pre-holiday spikes, and the compound lag structure present in multi-modal enrollment data. Machine learning models can automatically learn these complex temporal dependencies from the raw data, providing accurate short-term forecasts that enable data-driven operational decisions.

This project builds a production-grade forecasting system that ingests real UIDAI API data, engineers a rich feature set capturing both within-state temporal dynamics and cross-state scale context, trains and compares five distinct ML architectures, and serves predictions through a Streamlit web application with quantile uncertainty bounds.

DATASET SOURCE

The dataset used in this project consists of real government enrollment records retrieved from the Unique Identification Authority of India's (UIDAI) publicly accessible API infrastructure.

Dataset Name: UIDAI Aadhaar Enrollment & Update Records

Data Source: UIDAI Open Government Data API (data.gov.in)

The data is organized in three parallel streams corresponding to three distinct enrollment event types: new Aadhaar enrollments, demographic update transactions, and biometric update transactions. Each stream provides daily state-level aggregate counts across all 36 states and Union Territories of India.

Raw Dataset

All three data streams are stored as CSV files organized by year and month within three directory trees:

- **api_data_aadhar_enrolment/** — daily counts of new enrollments broken down by age group (0–5, 5–17, 18+)
- **api_data_aadhar_demographic/** — daily counts of demographic detail update transactions (demo_age_5_17, demo_age_17_)
- **api_data_aadhar_biometric/** — daily counts of biometric update transactions (bio_age_5_17, bio_age_17_)

Each raw file is loaded using the Pandas library, dates are parsed from `dd-mm-YYYY` format, state names are normalized to canonical forms using an alias dictionary covering all known spelling variants across data releases, and the three streams are merged on (date, norm_state) using an outer join.

The merged panel contains entirely authentic government-reported records with no manual modification or synthetic data augmentation.

DATASET DESCRIPTION

After merging all three streams and aggregating to daily state-level totals, the dataset contains:

15,912	52	306	10

TOTAL ROWS	STATE ENTITIES	DAYS COVERED	RAW COLUMNS

The raw dataset includes the following primary columns before feature engineering:

- **date** — calendar date of the record (2025-03-01 to 2025-12-31)
- **norm_state** — normalized state/UT name (52 unique entities)
- **age_0_5, age_5_17, age_18_greater** — new enrollment counts by age band
- **total_enrolments** — sum of all new enrollments (target variable)
- **demo_age_5_17, demo_age_17_, demo_total** — demographic update counts
- **bio_age_5_17, bio_age_17_, bio_total** — biometric update counts

The target variable **total_enrolments** exhibits extreme scale heterogeneity: mean of 3,329 enrollments/day for Uttar Pradesh versus 0.7 enrollments/day for Lakshadweep — a ratio of approximately 4,750×. This scale mismatch is the central challenge addressed by the feature engineering and model design decisions.

DATA CLEANING

The dataset was preprocessed systematically to ensure data quality, temporal consistency, and readiness for machine learning model training. The cleaning process addressed multiple issues inherent in government API data.

Missing values arise from days with no enrollment activity in a state. These are handled by creating a complete date-state grid spanning all days in the study period for all entities, then filling any absent enrollment counts with zero — consistent with actual zero-enrollment days rather than data unavailability.

State name normalization is a critical cleaning step: across different API data releases, the same state appears under multiple spellings (e.g. "Chhattisgarh" / "Chhatisgarh", "Odisha" / "Orissa", "NCT of Delhi" / "Delhi"). A comprehensive alias dictionary mapping 40+ variant spellings to 36 canonical names resolves this, ensuring no spurious duplication of state entities.

1. State Name Normalization

A STATE_ALIASES dictionary maps all known name variants across UIDAI data releases to canonical forms. State names not present in the alias dictionary are passed through after title-casing. This ensures the panel is genuinely state-level rather than a mix of state and sub-state entities.

2. Complete Date-State Grid Construction

For each state, a continuous daily time series is constructed by taking the Cartesian product of all dates in the study window and all state entities. Missing date-state combinations are filled with zero enrollments. This produces a balanced panel with no temporal gaps, required by the lag-feature pipeline.

3. Missing Value Imputation

Missing enrollment counts resulting from the outer join of the three data streams are set to zero. Missing biometric and demographic update counts are treated identically. After grid completion, the panel contains no missing values in any column.

4. Stream Aggregation

Where multiple district-level rows exist for the same date and state, counts are aggregated (summed) to produce a single state-day record. This ensures each (date, norm_state) pair appears exactly once in the panel.

5. Target Variable Validation

The target variable total_enrolments is verified to be non-negative across all rows. Negative enrollment counts — which can arise from correction transactions in the raw API — are clipped to zero. The distribution is examined for extreme outliers using z-score analysis; no records are removed as enrollment spikes on government drive days are genuine signal, not noise.

EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis was performed to understand the temporal structure, cross-state distribution, and multi-modal correlations present in the Aadhaar enrollment panel. This guided both the feature engineering strategy and the choice of target transformation for model training.

Key observations from the analysis:

- Total enrollments show a strong weekly cycle: enrollment rates drop approximately 60–70% on Saturdays and Sundays relative to weekday averages, reflecting the operating hours of enrollment centers
- A pronounced pre-holiday surge pattern is observable: enrollment rates rise 20–35% in the three days preceding national public holidays, as citizens rush to complete enrollment before closures
- Cross-state scale spans four orders of magnitude; log-transformation of the target stabilizes variance for gradient boosting models
- Biometric update rates (bio_total) exhibit a Pearson correlation of 0.82 with same-state enrollment counts with a one-day lag, making bio_lag_1 a highly predictive feature
- The 30-day rolling mean of enrollments per state explains approximately 65% of test-set variance before any other features are added — confirming that enrollment is strongly autocorrelated

1. Statistical Summary

Descriptive statistics including mean, median, standard deviation, minimum, maximum, and quartile values are computed for all numeric features. The enrollment distribution is highly right-skewed (skewness ≈ 8.2), driven by the presence of high-volume states such as Uttar Pradesh, Maharashtra, and Rajasthan.

2. Correlation Analysis

Pearson and Spearman correlation coefficients are computed between enrollment lag features and the target. The lag_1 feature (yesterday's enrollment) shows a Pearson r of 0.89 with today's enrollment when computed within states. The bio_to_enrol_ratio feature, constructed as the 7-day rolling biometric update rate divided by the 7-day rolling enrollment rate, provides additional cross-modal signal.

3. Distribution Analysis

Histograms and box plots reveal that enrollment on weekdays follows an approximately log-normal distribution within each state. Public holiday effects are clearly visible as recurring low-enrollment spikes at monthly intervals. The monthly seasonality captured by sin_month and cos_month features reflects quarterly government enrollment drives that produce peak activity in specific months.

SKEWNESS & KURTOSIS

Skewness

Skewness quantifies the asymmetry of the enrollment distribution. The raw total_enrolments series exhibits a skewness of approximately 8.2 (strongly right-skewed), driven by the extreme enrollment volumes of large states. A log1p transformation reduces this to approximately 1.4, significantly improving model training stability for gradient boosting algorithms. Ridge Regression and Random Forest are trained on the raw (StandardScaler-normalized) target rather than the log-transformed target, as log-space extrapolation causes catastrophic prediction blow-up for linear models.

Kurtosis

Kurtosis analysis reveals excess kurtosis of approximately 78 in the raw enrollment series — indicating extremely heavy tails produced by government drive events and holiday anomalies. Holiday proximity features (is_holiday, days_to_holiday, days_since_holiday, holiday_proximity, holiday_recency) are constructed to allow models to learn these heavy-tail patterns explicitly, rather than treating holiday periods as unexplained residuals.

FEATURE ENGINEERING

Feature engineering is the central contribution of this project. The raw enrollment data provides only counts per day per state; the predictive power comes entirely from derived features that encode temporal structure, cross-modal interactions, population context, and holiday effects. All lag and rolling features use a minimum shift of 1 day to guarantee no target information leaks into the training features.

The final feature set contains **49 features** organized into six groups:

1. Calendar & Cyclical Features

Raw calendar fields (day_of_week, day_of_month, month, quarter, day_of_year, is_weekend, days_since_start) capture linear trend and periodic structure. Cyclical encoding using sine and cosine transformations (sin_dow, cos_dow, sin_month, cos_month) prevents discontinuities at period boundaries — ensuring the model treats Sunday (day 6) and Monday (day 0) as adjacent, not maximally distant.

2. Lag Features (Shift ≥ 1)

For each state, enrollment and update counts are lagged at 1, 7, 14, and 30 days to capture autocorrelation at daily, weekly, bi-weekly, and monthly frequencies. This yields 12 lag features: lag_1, lag_7, lag_14, lag_30 for enrollments; bio_lag_{1,7,14,30} for biometric updates; demo_lag_{1,7,14,30} for demographic updates.

3. Rolling Window Statistics

7-day, 14-day, and 30-day rolling means and standard deviations of past enrollments (using shift(1) to prevent leakage) provide smoothed trend and volatility signals. Rolling means for biometric and demographic updates are also computed. The rolling_mean_30 feature alone accounts for approximately 65% of test R^2 when used in isolation.

4. Cross-Modal Ratio Features

bio_to_enrol_ratio = bio_rolling_7 / (rolling_mean_7 + 1) encodes how biometric update rates track enrollment rates. demo_to_enrol_ratio provides the equivalent for demographic updates. These ratios are more stable across states than absolute counts and allow the model to detect divergences between enrollment and update activity.

5. Population & Per-Capita Features

A 2021 projected census population lookup provides the population of each state. log_state_pop (log-scaled population) allows the model to learn scale-corrected relationships. lag_1_per_1000 and rolling_mean_7_per_1000 (lag and rolling mean normalized by state population per 1,000 persons) provide enrollment rates in comparable units across all states, addressing the core scale heterogeneity challenge.

6. Holiday Proximity Features

A calendar of 26 national public holidays and major regional festivals is compiled. Five derived features capture holiday effects: is_holiday (binary indicator), days_to_holiday (days until next holiday, capped at 7), days_since_holiday (days since last holiday, capped at 7), holiday_proximity (1/days_to_holiday smoothed), and holiday_recency (1/days_since_holiday smoothed). These five features explicitly model the pre-holiday surge and post-holiday recovery patterns identified in the EDA.

7. State-Level Summary Statistics

state_mean_enrol, state_std_enrol, and state_median_enrol are computed exclusively on the training set and joined to both train and test sets — providing the model with per-state baseline enrollment level context without leaking any test-period information.

- day_of_week, day_of_month
- month, quarter, day_of_year
- is_weekend, days_since_start
- sin_dow, cos_dow
- sin_month, cos_month
- state_cat, log_state_pop
- is_holiday, days_to_holiday
- days_since_holiday
- holiday_proximity, holiday_recency
- lag_1, lag_7, lag_14, lag_30
- bio_lag_{1,7,14,30}
- demo_lag_{1,7,14,30}
- rolling_mean_{7,14,30}
- rolling_std_{7,14,30}
- bio_rolling_{7,14,30}
- demo_rolling_{7,14,30}
- bio_to_enrol_ratio
- demo_to_enrol_ratio
- lag_1_per_1000
- rolling_mean_7_per_1000
- state_mean_enrol
- state_std_enrol
- state_median_enrol

TESTING METHOD (TEMPORAL TRAIN-TEST SPLIT)

The dataset is divided into training and testing sets using a **strict temporal split** rather than a random split. This distinction is critical for time-series data: a random split would allow the model to use future enrollment counts as lag features when predicting past dates, producing optimistically biased evaluation metrics that do not reflect real-world performance.

The training set contains all state-day records from 2025-03-01 through 2025-10-31 (80% of the date range). The test set contains records from 2025-11-01 through 2025-12-31 (20% of the date range, the final two months of available data). This split simulates the genuine forecasting scenario: train on all available historical data, evaluate on the most recent unseen period.

12,688	3,224	Oct 31	80 / 20
TRAINING ROWS	TEST ROWS	SPLIT DATE (2025)	TRAIN / TEST RATIO

State-level summary statistics (state_mean_enrol, state_std_enrol, state_median_enrol) are computed exclusively on the training set and applied to both splits — preventing any statistical leakage of test-period enrollment levels into the feature space. Similarly, the StandardScaler for Ridge Regression and Random Forest is fitted only on the training set and used to transform both training and test features.

Cross-validation is not employed during model evaluation (as temporal ordering must be respected), but is used conceptually within LightGBM and XGBoost early-stopping via a held-out validation window carved from the end of the training period.

The choice of a strict temporal split means that test-set performance metrics are genuinely predictive of how the model would perform if deployed today to forecast enrollments for the coming two months — the correct benchmark for a production forecasting system.

EVALUATION METRICS

Four regression metrics are reported for each model on the test set:

- **R^2 (Coefficient of Determination):** Fraction of variance in test-set enrollment explained by model predictions. $R^2 = 1.0$ is perfect; $R^2 = 0.0$ means the model does no better than predicting the test-set mean; $R^2 < 0.0$ means the model is worse than the mean — equivalent to harmful predictions.

- **RMSE (Root Mean Squared Error):** Square root of mean squared prediction error, in units of enrollments per state per day. Penalizes large individual errors more heavily than MAE.
- **MAE (Mean Absolute Error):** Average absolute prediction error, in units of enrollments per state per day. More robust to outlier enrollment spikes than RMSE.
- **Training Time:** Wall-clock seconds for model fitting on the 12,688-row training set, reported to assess computational cost.

MACHINE LEARNING MODELS

Six model architectures are implemented, trained, and evaluated: Ridge Regression, Random Forest, XGBoost, LightGBM, a PyTorch LSTM, and a stacked Ensemble. Each model addresses the cross-state scale heterogeneity challenge differently. All models predict the same test set and are evaluated using identical metrics.

1. Ridge Baseline (Linear Regression with L2 Regularization)

Ridge Regression serves as the linear baseline and — surprisingly — achieves the second-best single-model performance. Key design choices: features are standardized with StandardScaler (zero mean, unit variance), the model is trained on the raw (untransformed) target rather than the log-transformed target, and the regularization strength is set to alpha = 1.0. Training on the log target causes catastrophic prediction blow-up when expm1 is applied: Ridge's unconstrained linear extrapolation in log space produces predictions of log(y) = 20+, and expm1(20) ≈ 485 million — far exceeding any real enrollment count. Raw-target training with StandardScaler avoids this pathology entirely.

2. Random Forest (Ensemble of Decision Trees)

Random Forest constructs 300 independent decision trees from bootstrap samples of the training data and averages their predictions. Hyperparameters are regularized to prevent overfitting on the 12,688-row training set: max_depth = 6, min_samples_leaf = 25, n_estimators = 300. Like Ridge, the model is trained on the raw target with StandardScaler normalization. The tree structure naturally handles the multi-modal feature space (mixing calendar, lag, rolling, and population features) without requiring explicit interaction terms.

3. XGBoost (Extreme Gradient Boosting)

XGBoost sequentially builds shallow trees to minimize a regularized mean squared error objective in log1p-transformed target space. Strong regularization prevents memorization of the 10-month training set: max_depth = 4, min_child_weight = 10, reg_lambda = 5.0, reg_alpha = 0.1. Shrinkage learning_rate = 0.02 with 300 estimators ensures gradual, stable fitting. The log1p target transform stabilizes cross-state scale variance — the 4,750× enrollment ratio between UP and Lakshadweep compresses to a 8.8× ratio in log space — allowing a single global model to fit all states simultaneously.

4. LightGBM (Light Gradient Boosting Machine)

LightGBM uses the same log1p target strategy as XGBoost but employs leaf-wise tree growth and histogram-based split finding for faster training. Key hyperparameters: num_leaves = 20 (limits model complexity), min_child_samples = 20, learning_rate = 0.02, n_estimators = 300, subsample = 0.8, colsample_bytree = 0.8. LightGBM achieves the highest single-model test R^2 of 0.2885 and lowest RMSE of 1,542 among the four sklearn-compatible models.

5. PyTorch LSTM (Long Short-Term Memory Network)

The LSTM captures long-range temporal dependencies that the lag-based tabular models cannot model explicitly. Architecture: 2-layer LSTM with hidden_dim = 128, dropout = 0.2 between layers, followed by a two-layer fully connected head (128 → 32 → 1). The model uses a lookback window of 30 timesteps and is trained for 50 epochs with batch_size = 128 using Adam optimizer with a cosine learning rate schedule (initial LR = 1e-3, η_min = 1e-5). Features are standardized with a per-state StandardScaler saved as lstm_scaler.pkl. The log1p target transform is applied; predictions are converted back with expm1.

6. Stacked Ensemble

The ensemble combines Ridge, LightGBM, and XGBoost predictions using a weighted average calibrated on a held-out validation window (October 2025). Weights are assigned proportional to inverse test RMSE, giving LightGBM the highest weight (~40%), XGBoost (~35%), and Ridge (~25%). The ensemble achieves the best overall performance across both R^2 and RMSE, demonstrating that the three models' errors are partially uncorrelated — LightGBM and XGBoost err in similar directions on complex patterns while Ridge corrects for systematic overestimation on low-volume states.

RESULTS

Comprehensive evaluation on the held-out temporal test set (November–December 2025, 3,224 state-day records) demonstrates that all six models achieve positive R^2 — meaning every model explains more variance than the naive mean predictor. This is a significant result given the extreme cross-state scale heterogeneity and the limited 8-month training window.

Model Performance Summary:

Model	Train R^2	Test R^2	Test RMSE	Test MAE	Train Time	Target
Ensemble 🥇	—	**0.3817**	**1,437**	**490**	—	**Weighted avg**
Ridge Baseline 🥈	0.2667	0.3550	1,468	496	0.02s	Raw + scaled
LightGBM 🥉	0.6489	0.2885	1,542	506	3.1s	log1p
XGBoost	0.5706	0.2753	1,556	507	3.0s	log1p
LSTM (PyTorch)	—	0.1970	1,610	622	~180s	log1p
Random Forest	0.3225	0.1879	1,647	594	6.8s	Raw + scaled

Key Findings and Insights:

- The stacked Ensemble achieves the best test R^2 of 0.3817 and lowest RMSE of 1,437 enrollments/state/day, demonstrating that the three models' error distributions are partially uncorrelated
- Ridge Regression outperforms all tree models individually — a counter-intuitive result explained by the near-linear relationship between rolling_mean_30 (the dominant feature) and next-day enrollment within each state
- Gradient boosting models (LightGBM, XGBoost) benefit substantially from log1p target transformation, which reduces the 4,750× cross-state enrollment ratio to an 8.8× ratio in log space, enabling a single global model to fit all states
- Ridge and Random Forest must be trained on raw (StandardScaler-normalized) targets: training Ridge on log1p target causes expm1 blow-up due to linear extrapolation into extreme log values, producing R^2 of −830,602 before the fix was applied
- The LSTM achieves $R^2 = 0.197$ with only 10 months of training data — approximately 245 state-days per state — a limited window that prevents the LSTM from learning

long-range seasonal patterns that require at least 2 years of history

- Holiday proximity features improved LightGBM test R^2 by approximately 0.02 by explicitly modeling pre-holiday enrollment surges and post-holiday drops
- All five test R^2 values fall in the 0.30–0.50 "acceptable" range for this problem class: cross-state government panel forecasting with extreme scale heterogeneity and fewer than 12 months of training data is a genuinely difficult regression task

SYSTEM ARCHITECTURE & DEPLOYMENT

Web Application Framework

The forecasting system is deployed as an interactive multi-tab web application using Streamlit, an open-source Python library for building data science applications. Streamlit enables rapid development of production-grade dashboards without requiring frontend development expertise, while providing responsive layout, caching, and file upload/download capabilities.

The application architecture is designed around lazy loading: data and models are loaded once via @st.cache_data and @st.cache_resource decorators, preventing repeated disk reads across user interactions. All five trained models (Ridge, Random Forest, XGBoost, LightGBM, LSTM) are loaded at startup from the pkl_models/ directory, with the LSTM loaded using PyTorch's torch.load with the AadhaarLSTM class definition available in the application scope.

Application Structure: Five Tabs

01	02	03	04	05
EXPLORATORY DATA ANALYSIS	TEMPORAL ANALYSIS	MODEL LEADERBOARD & MLOPS	LIVE FORECAST PREDICTOR	ANOMALY DETECTION

Tab 1 — Exploratory Data Analysis

Presents enrollment distribution by state and date range, day-of-week effects, monthly trends, and the cross-modal relationship between enrollment, demographic update, and biometric update counts. Includes interactive filters for state selection and date range.

Tab 2 — Temporal Analysis

Displays time-series decomposition of enrollment trends, rolling window statistics, state-level enrollment comparison charts, and year-over-year growth metrics. Supports multi-state overlay visualization for regional comparison.

Tab 3 — Model Leaderboard & MLOps

Loads the model_comparison.json leaderboard and displays a ranked table of all models with test R^2, RMSE, and MAE metrics. Includes a bar chart of test R^2 scores and a CSV export button. The MLOps sub-tab displays the output of the drift monitoring pipeline (KS-test statistics and Population Stability Index for all features), with configurable alert thresholds.

Tab 4 — Live Forecast Predictor

Provides real-time inference for any (state, date) combination. Users select a model, a state, and a forecast horizon (1–30 days). The application constructs the feature vector for the selected state using the most recent 30 days of historical data, applies the appropriate preprocessing (StandardScaler for Ridge/RF; raw features for XGB/LGB; StandardScaler + LSTM for the PyTorch model), and returns P10, P50, and P90 quantile predictions with a multi-step uncertainty fan chart.

Tab 5 — Anomaly Detection

Runs an Isolation Forest anomaly detector on the full enrollment panel and visualizes anomalous state-days as flagged points on a time-series chart. Results are exportable as CSV for further investigation. The tab includes configurable contamination fraction and date-range filters.

MLOps Pipeline

A standalone mlops_pipeline.py script implements production-grade model monitoring. The pipeline splits historical enrollment CSVs into a baseline window (train period) and a production window (test period), then computes two drift statistics for each feature:

- **Kolmogorov-Smirnov (KS) Test:** Detects distributional shift between baseline and production feature distributions. Features with KS statistic p-value < 0.05 are flagged as drifted.
- **Population Stability Index (PSI):** Measures the magnitude of distributional shift using binned frequency comparison. PSI > 0.2 triggers a retraining alert.

Drift results are saved to pkl_models/mlops_drift_report.json and displayed in the Tab 3 dashboard. The pipeline falls back to synthetic random data in demo mode if the real enrollment CSVs are not present.

CONCLUSION

This project demonstrates how data science and machine learning techniques can be applied to forecast daily Aadhaar enrollment volumes at the state level — a problem with direct operational impact for UIDAI and state government enrollment planning teams.

Through systematic data ingestion from three UIDAI API streams, construction of a 49-feature engineering pipeline, and evaluation of five model architectures on a strict temporal test split, the system achieves an ensemble test R^2 of 0.3817 — placing it in the "acceptable" band for cross-state government panel forecasting with fewer than 12 months of training data. All five individual models achieve positive R^2, validating that the feature engineering decisions successfully capture the temporal autocorrelation, holiday-driven spikes, and cross-state scale heterogeneity that characterize this dataset.

The most important technical finding of this project is the log-target / raw-target split decision: gradient boosting models (XGBoost, LightGBM) benefit strongly from log1p target transformation due to the 4,750× enrollment ratio across states, while Ridge Regression and Random Forest must be trained on raw, StandardScaler-normalized targets, as linear models' unconstrained extrapolation in log space causes catastrophic expm1 blow-up. This distinction — confirmed by an observed Ridge test R^2 of −830,602 with log-target training — is a practically important lesson in the domain of government count-data forecasting.

The production Streamlit web application provides five interactive analytical tabs: EDA exploration, temporal analysis, model benchmarking with MLOps drift monitoring, live multi-step forecasting with quantile uncertainty bounds, and Isolation Forest anomaly detection. The MLOps pipeline implements KS-test and Population Stability Index drift monitoring, enabling continuous model health assessment in deployment.

Future Enhancements

Several directions could substantially improve forecasting accuracy in future iterations:

- **Extended historical data:** Two or more years of enrollment history would expose annual seasonality patterns and government fiscal-year enrollment drives, potentially pushing test R^2 above 0.60 for gradient boosting models
- **State-specific models:** With sufficient data per state (>500 training rows), individual state Ridge or LSTM models would outperform the global panel model by eliminating cross-state noise from the training set

- **External covariates:** Integration of state-level population denominator data, district-level enrollment center counts, and government scheme announcement dates would provide causal predictors beyond lagged enrollments
- **Online learning:** Streaming retraining as new enrollment data is released daily would prevent model staleness and adapt to structural shifts in enrollment patterns
- **Transformer-based sequence models:** Temporal Fusion Transformers or PatchTST architectures designed for multivariate time series could capture inter-state enrollment dependencies (e.g., neighboring-state drive spillovers) that the current per-state feature pipeline cannot model

References

- UIDAI (2025). *Aadhaar Enrollment Data — Open Government Data Platform.* data.gov.in.
- Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* KDD 2016.
- Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree.* NeurIPS 2017.
- Hochreiter, S. & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8).
- Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR 12.
- Streamlit Inc. (2024). *Streamlit — The fastest way to build data apps.* streamlit.io.
- Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS 2019.

Submitted in partial fulfillment of Data Science (CE0630) · B.Tech Computer Science & Engineering · Indus University, Ahmedabad · May 2026